

N-ary Trees, Balanced Trees and Search Trees

Shahriar Ivan

Lecturer, Department of CSE, IUT



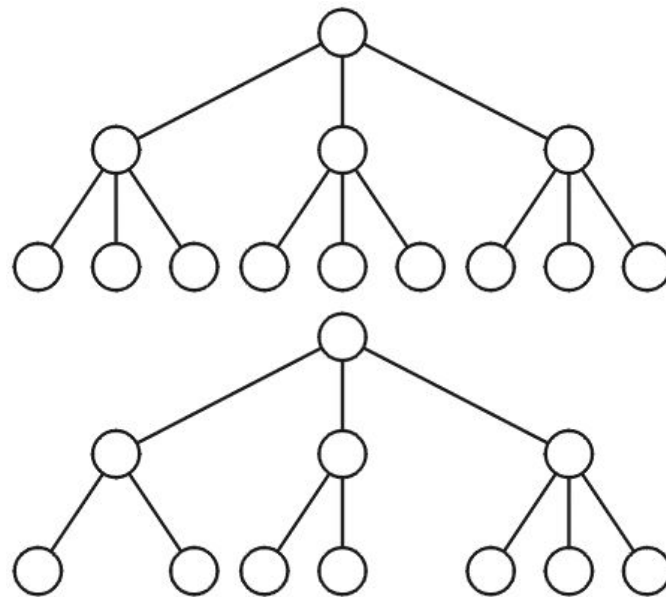
N-ary Trees

One generalization of binary trees are a class of trees termed N-ary trees:

- A tree where each node had N sub-trees, any of which may be empty trees

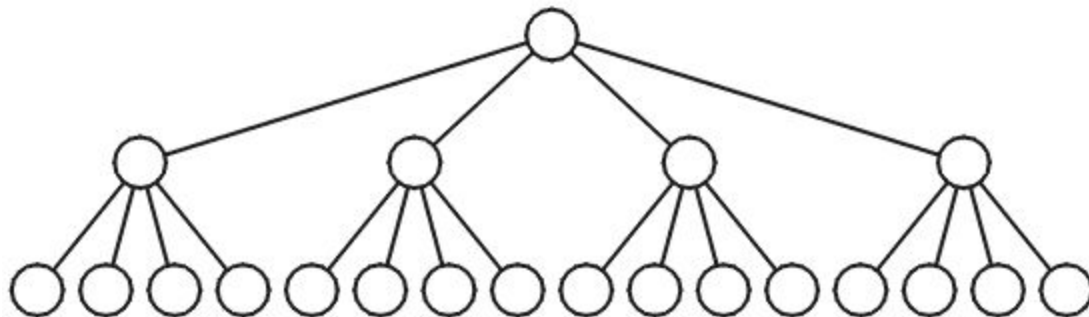
Examples of a ternary (3-ary) trees

- We don't usually indicate empty sub-trees



N-ary Trees

Example of a perfect quaternary (4-ary) tree



Under certain conditions, the number of operations required may be slightly fewer with quaternary trees (e.g., with heaps)

- The asymptotic behaviour is similar

Perfect N-ary Trees

Each node can have N children

The number of nodes in a perfect N-ary tree of height h is

$$1 + N + N^2 + N^3 \dots + N^h$$

This is a geometric sum, and therefore, the number of nodes is

$$n = \sum_{k=0}^h N^k = \frac{N^{h+1} - 1}{N - 1}$$

N-ary Trees

Recall that the height for perfect binary tree is approximately $\lg(n)$

Therefore, if the height of an N-ary tree is $\log_N(n)$, the ratio of the heights is:

$$\frac{\log_2(n)}{\log_N(n)} = \frac{\log_2(n)}{\log_2(n)/\log_2(N)} \\ = \log_2(N)$$

N-ary Trees

Therefore:

- The height of a perfect quaternary tree is approximately $\frac{1}{2}$ the height of a perfect binary tree with approximately the same number of nodes
- The height of a perfect 16-ary tree is approximately $\frac{1}{4}$ the height of a perfect binary tree with the same number of nodes

Balanced Trees

We know, run times depend on the height of the trees

As was noted in the previous section on Binary Trees:

- The best case height is $\Theta(\lg(n))$
- The worst case height is $\Theta(n)$

The average height of a randomly generated binary search tree is actually $\Theta(\ln(n))$

However, following random insertions and erases, the average height tends to increase to $\Theta(\sqrt{n})$

Requirement for Balance

We want to ensure that the run times never fall into $\omega(\lg(n))$

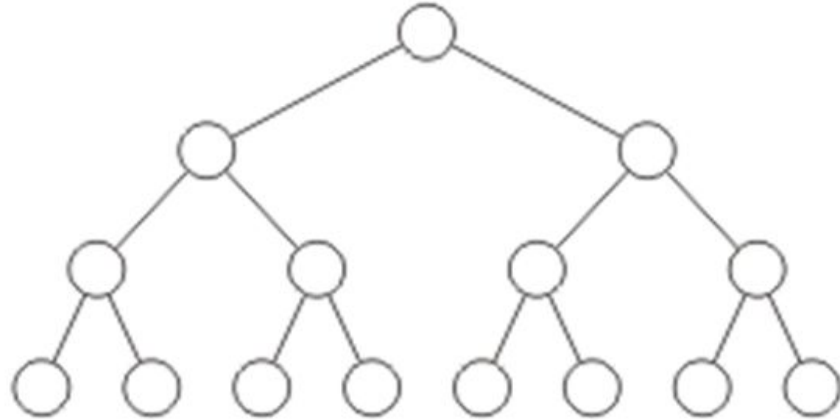
Requirement:

We must maintain a height which is $\Theta(\lg(n))$

To do this, we will define an idea of balance

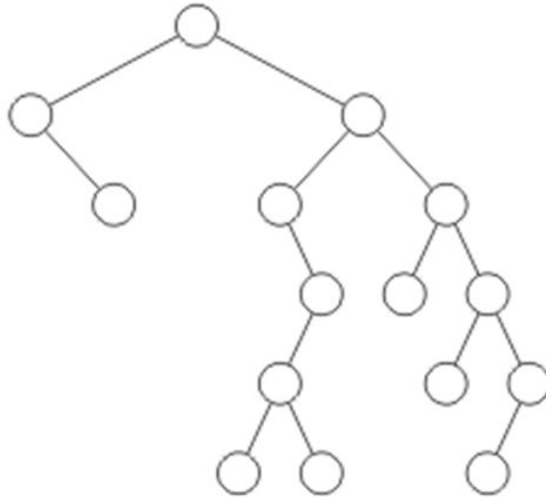
Example of balance

- For a perfect tree, all nodes have the same number of descendants on each side
- Perfect binary trees are balanced while linked lists are not



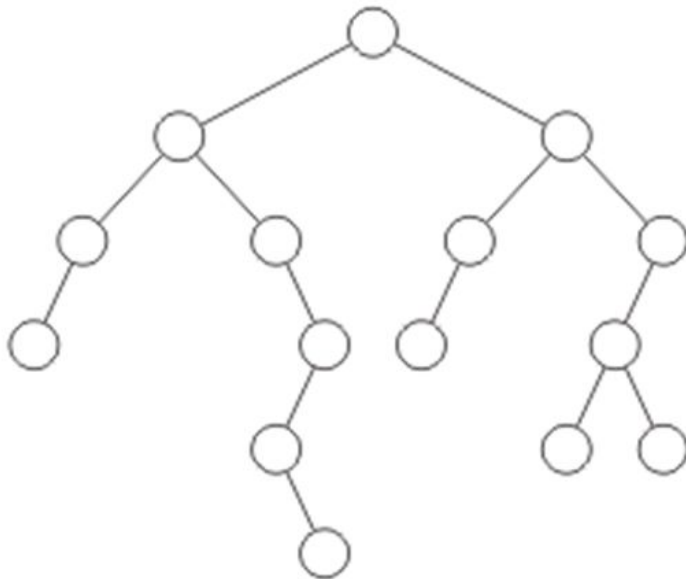
Example of balance

- This binary tree would also probably not be considered to be “balanced” at the root node



Example of balance

- How about this example?
 - The root seems balanced, but what about the left sub-tree?



Definition for Balance

We must develop a quantitative definition of balance which can be applied

Balanced may be defined by:

- Height balancing: comparing the heights of the two sub trees
- Null-path-length balancing: comparing the null-path-length of each of the two sub-trees (the length to the closest null sub-tree/empty node)
- Weight balancing: comparing the number of null sub-trees in each of the two sub trees

It can be mathematically prove that if a tree satisfies the definition of balance, its height is $\Theta(\lg(n))$

Definition of Balance

We will see one definition of height balancing:

- AVL trees

We will also look at B+ trees

- Balanced trees, but not binary trees

Binary Search Trees

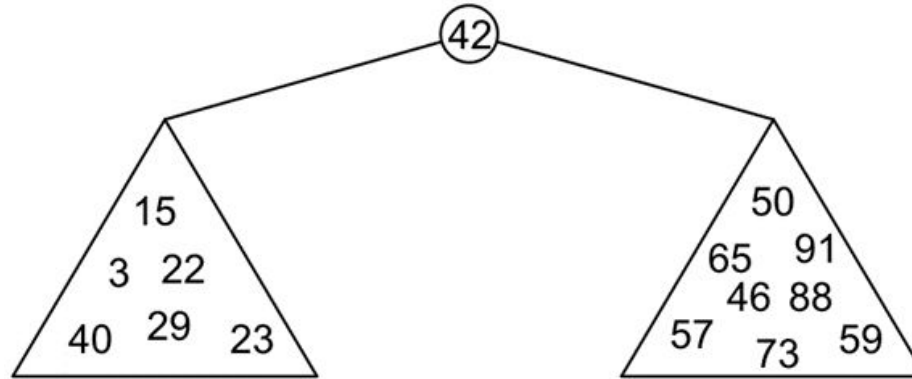
Recall that with a binary tree, we can dictate an order on the two children

We will exploit this order:

- Require all objects in the left sub-tree to be less than the object stored in the root node, and
- Require all objects in the right sub-tree to be greater than the object in the root object

Binary Search Trees

Graphically, we may show the relationship



Each of the two sub-trees will themselves be binary search trees

Binary Search Trees

Notice that we can already use this structure for searching:

- examine the root node and if we have not found what we are looking for:
 - If the object is less than what is stored in the root node, continue searching in the left sub-tree
 - Otherwise, continue searching the right sub-tree

With a linear order, one of the following three must be true:

$a < b$ $a = b$ $a > b$

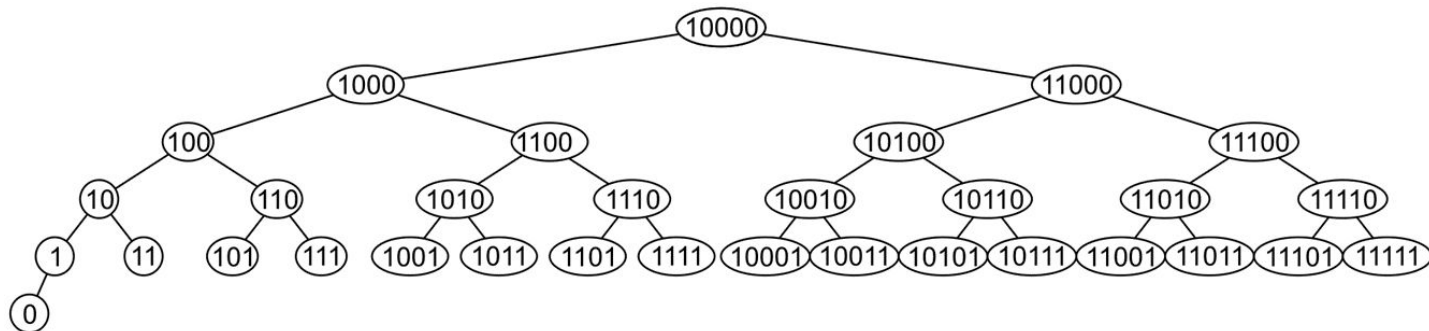
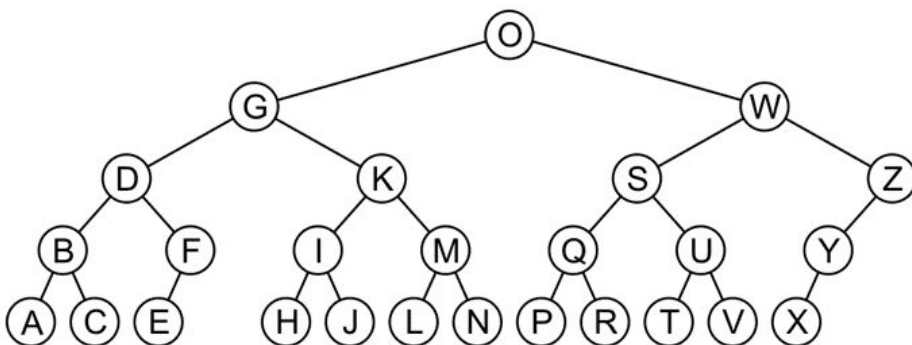
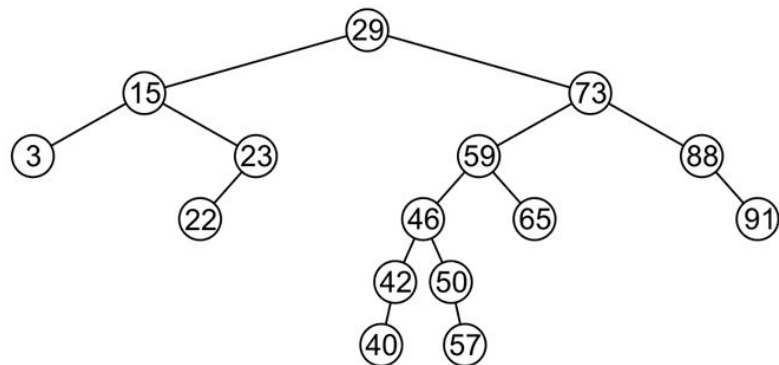
Definition of BST

Thus, we define a non-empty binary search tree as a binary tree with the following properties:

- The left sub-tree (if any) is a binary search tree and all values are less than the root value, and
- The right sub-tree (if any) is a binary search tree and all values are greater than the root value

Examples

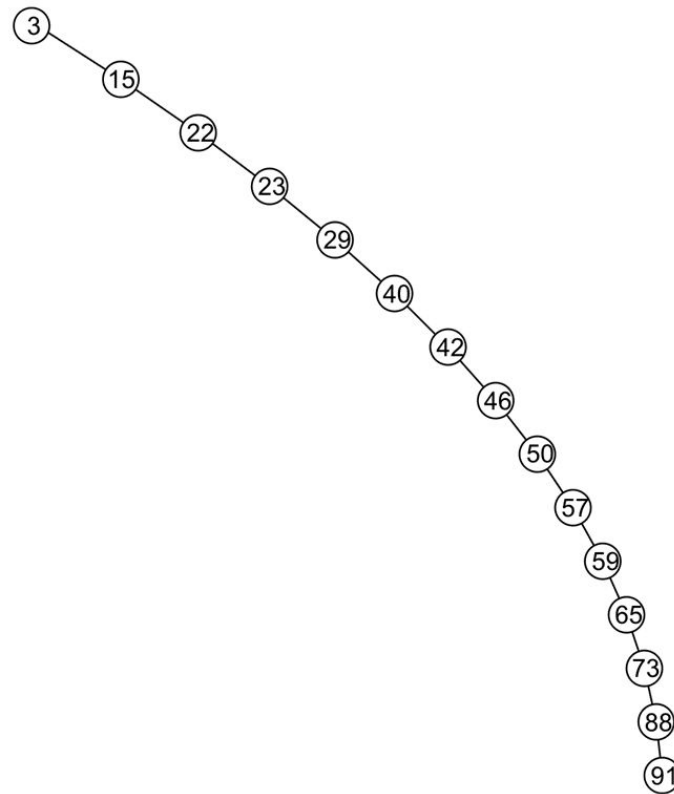
Here are other examples of binary search trees:



Examples

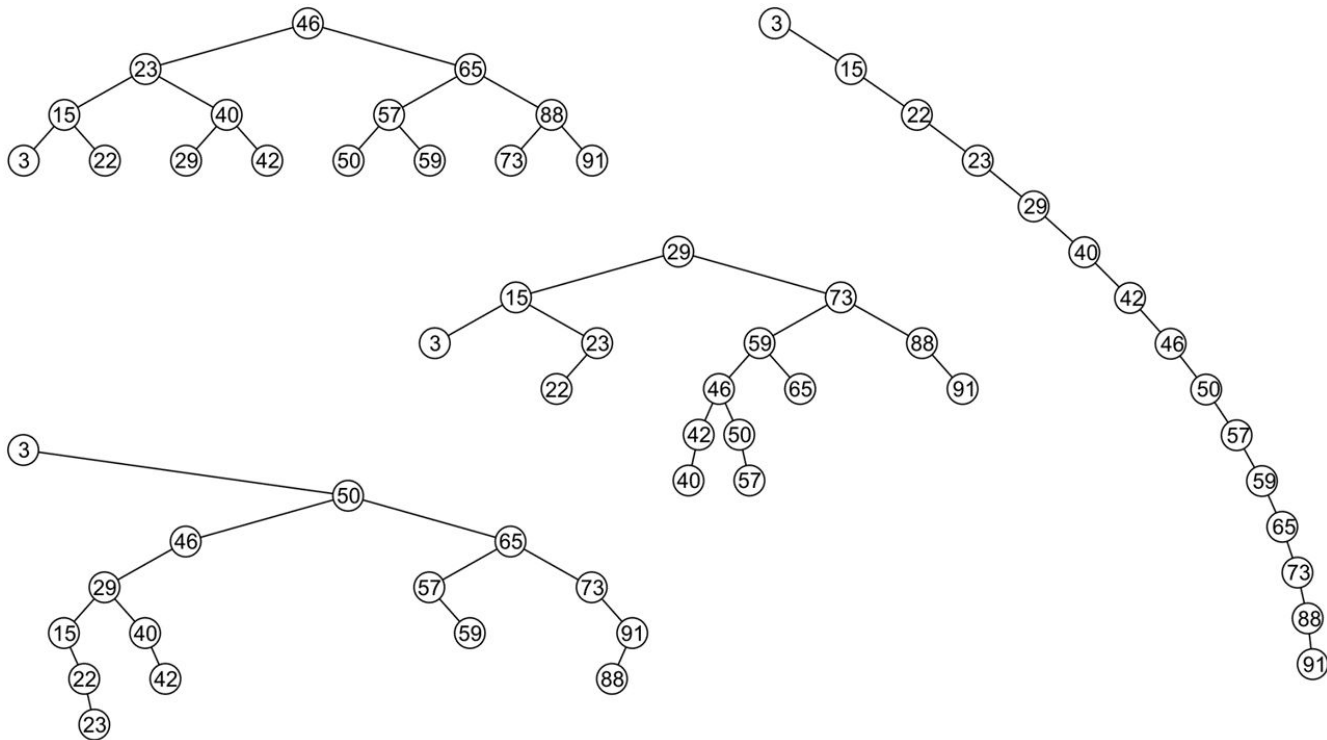
Unfortunately, it is possible to construct degenerate binary search trees

The following example is equivalent to a linked list, i.e., $O(n)$



Examples

All these binary search trees store the same data



Duplicate Values

We will assume that in any binary tree, we are not storing duplicate values unless otherwise stated

- In reality, it is seldom the case where duplicate values in a container must be stored as separate entities

You can always consider duplicate values with modifications to the algorithms we will cover